

机器人自动走迷宫

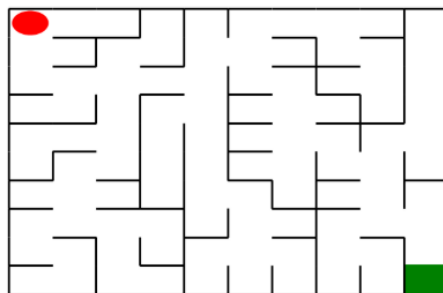
实验目的

1. 掌握迷宫环境下的路径规划算法，包括深度优先搜索（DFS）算法的实现和应用。
2. 探索强化学习在复杂环境中的应用，尤其是利用深度强化学习（DQN）算法解决机器人路径规划问题。
3. 通过对比传统算法和强化学习方法，理解不同算法的优缺点。
4. 学习基于 PyTorch 深度学习框架的机器人训练与路径规划实现。

实验介绍

实验内容

在本实验中，要求分别使用基础搜索算法和 Deep QLearning 算法，完成机器人自动走迷宫。



如上图所示，左上角的红色椭圆既是起点也是机器人的初始位置，右下角的绿色方块是出口。游戏规则为：从起点开始，通过错综复杂的迷宫，到达目标点(出口)。

- 在任一位置可执行动作包括：向上走 'u'、向右走 'r'、向下走 'd'、向左走 'l'。
- 执行不同的动作后，根据不同的情况会获得不同的奖励，具体而言，有以下几种情况。
 - 撞墙
 - 走到出口
 - 其余情况
- 需要您分别实现**基于基础搜索算法**和 **Deep QLearning 算法**的机器人，使机器人自动走到迷宫的出口。

实验要求

- 使用 Python 语言。
- 使用基础搜索算法完成机器人走迷宫。
- 使用 Deep QLearning 算法完成机器人走迷宫。
- 算法部分需要自己实现，不能使用现成的包、工具或者接口。

实验环境

可以使用 Python 实现基础算法的实现，使用 Keras、PyTorch等框架实现 Deep QLearning 算法。

注意事项

- Python 与 Python Package 的使用方式，可在右侧 [API文档](#) 中查阅。
- 当右上角的『Python 3』长时间指示为运行中的时候，造成代码无法执行时，可以重新启动 Kernel 解决（左上角『Kernel』 - 『Restart Kernel』）。

参考资料

- 强化学习入门MDP: <https://zhuanlan.zhihu.com/p/25498081>
- QLearning 示例: <http://mnemstudio.org/path-finding-q-learning-tutorial.htm>
- QLearning 知乎解释: <https://www.zhihu.com/question/26408259>
- DeepQLearning 论文: <https://files.momodel.cn/Playing%20Atari%20with%20Deep%20Reinforcement%20Learning.pdf>

实验内容

Maze类

创建迷宫

通过迷宫类 Maze 可以随机创建一个迷宫。

1. 使用 Maze(maze_size=size) 来随机生成一个 size * size 大小的迷宫。
2. 使用 print() 函数可以输出迷宫的 size 以及画出迷宫图
3. 红色的圆是机器人初始位置
4. 绿色的方块是迷宫的出口位置

重要的成员方法

在迷宫中已经初始化一个机器人，你要编写的算法实现在给定条件下控制机器人移动至目标点。

Maze 类中重要的成员方法如下：

1. sense_robot()：获取机器人在迷宫中目前的位置。

return：机器人在迷宫中目前的位置。

2. move_robot(direction)：根据输入方向移动默认机器人，若方向不合法则返回错误信息。

direction：移动方向，如："u"，合法值为：['u', 'r', 'd', 'l']

return：执行动作的奖励值

3. can_move_actions(position)：获取当前机器人可以移动的方向

position：迷宫中任一处的坐标点

return：该点可执行的动作，如：['u','r','d']

4. is_hit_wall(self, location, direction)：判断该移动方向是否撞墙

location, direction: 当前位置和要移动的方向, 如(0,0), "u"

return: True(撞墙) / False(不撞墙)

5. draw_maze(): 画出当前的迷宫

强化学习算法

强化学习作为机器学习算法的一种, 其模式也是让智能体在“训练”中学到“经验”, 以实现给定的任务。但不同于监督学习与非监督学习, 在强化学习的框架中, 我们更侧重通过智能体与环境的**交互**来学习。通常在监督学习和非监督学习任务中, 智能体往往需要通过给定的训练集, 辅之以既定的训练目标 (如最小化损失函数), 通过给定的学习算法来实现这一目标。

然而在强化学习中, 智能体则是通过其与环境交互得到的奖励进行学习。

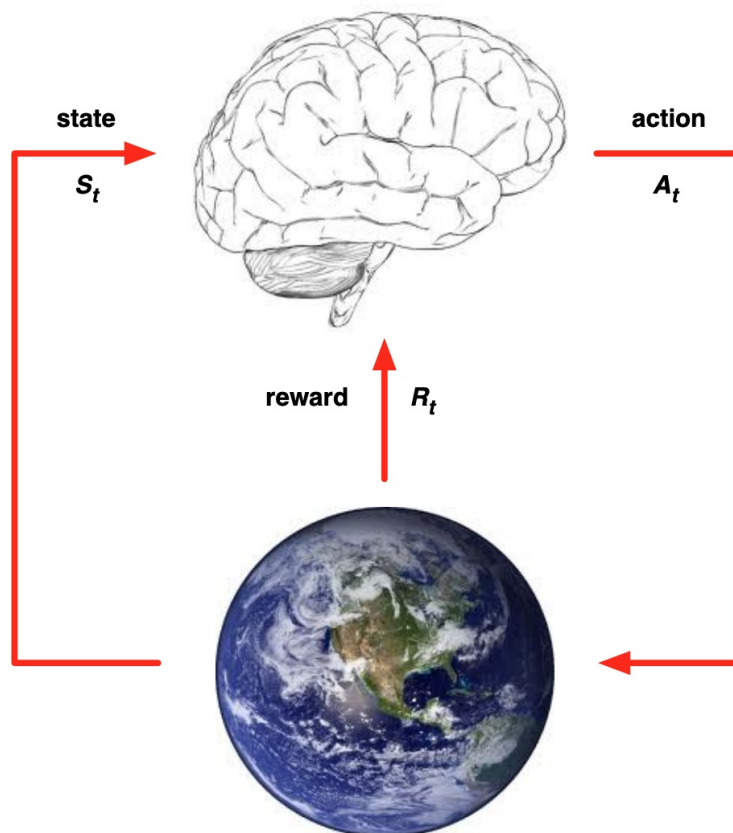
这个环境可以是虚拟的 (如虚拟的迷宫), 也可以是真实的 (自动驾驶汽车在真实道路上收集数据)。

在强化学习中有五个核心组成部分, 它们分别是: **环境 (Environment)**、**智能体 (Agent)**、**状态 (State)**、**动作 (Action)** 和**奖励 (Reward)**。

在某一时间节点 t :

- 智能体在从环境中感知其所处的状态 s_t
- 智能体根据某些准则选择动作 a_t
- 环境根据智能体选择的动作, 向智能体反馈奖励 r_{t+1}

通过合理的学习算法, 智能体将在这样的问题设置下, 成功学到一个在状态 s_t 选择动作 a_t 的策略 $\pi(s_t) = a_t$ 。



QLearning 算法

Q-Learning 是一个值迭代 (Value Iteration) 算法。

与策略迭代 (Policy Iteration) 算法不同，值迭代算法会计算每个“状态”或是“状态-动作”的值 (Value) 或是效用 (Utility)，然后在执行动作的时候，会设法最大化这个值。

因此，对每个状态值的准确估计，是值迭代算法的核心。

通常会考虑**最大化动作的长期奖励**，即不仅考虑当前动作带来的奖励，还会考虑动作长远的奖励。

Q 值的计算与迭代

Q-learning 算法将状态 (state) 和动作 (action) 构建成一张 Q_table 表来存储 Q 值，Q 表的行代表状态 (state)，列代表动作 (action)：

Q-Table	a_1	a_2
s_1	$Q(s_1, a_1)$	$Q(s_1, a_2)$
s_2	$Q(s_2, a_1)$	$Q(s_2, a_2)$
s_3	$Q(s_3, a_1)$	$Q(s_3, a_2)$

在 Q-Learning 算法中，将这个长期奖励记为 Q 值，其中会考虑每个“状态-动作”的 Q 值，具体而言，它的计算公式为：

$$Q(s_t, a) = R_{t+1} + \gamma \times \max_a Q(a, s_{t+1})$$

也就是对于当前的“状态-动作” (s_t, a) ，考虑执行动作 a 后环境奖励 R_{t+1} ，以及执行动作 a 到达 s_{t+1} 后，执行任意动作能够获得的最大的 Q 值 $\max_a Q(a, s_{t+1})$ ， γ 为折扣因子。

计算得到新的 Q 值之后，一般会使用更为保守地更新 Q 表的方法，即引入松弛变量 $alpha$ ，按如下的公式进行更新，使得 Q 表的迭代变化更为平缓。

$$Q(s_t, a) = (1 - \alpha) \times Q(s_t, a) + \alpha \times (R_{t+1} + \gamma \times \max_a Q(a, s_{t+1}))$$

机器人动作的选择

在强化学习中，**探索-利用** 问题是非常重要的问题。

具体来说，根据上面的定义，会尽可能地让机器人在每次选择最优的决策，来最大化长期奖励。

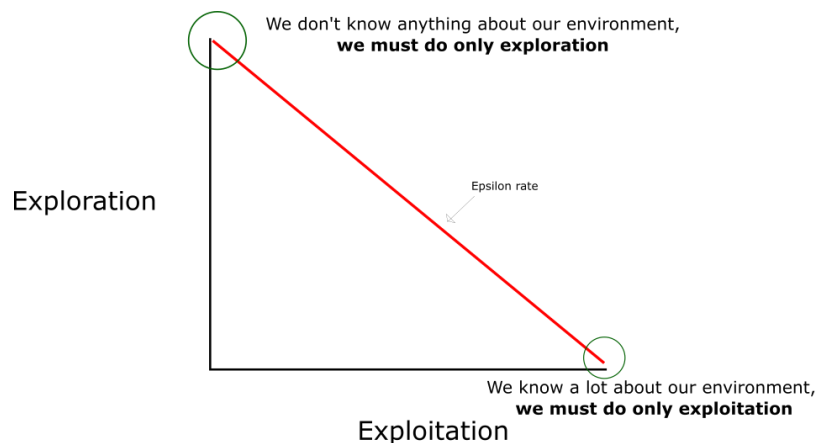
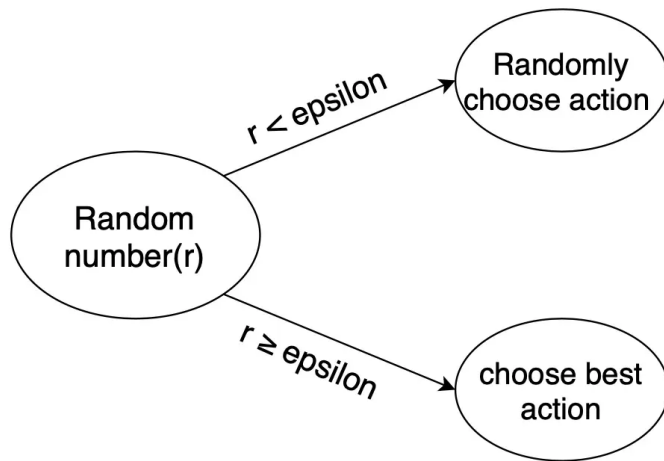
但是这样做有如下的弊端：

1. 在初步的学习中，Q 值是不准确的，如果在这个时候都按照 Q 值来选择，那么会造成错误。
2. 学习一段时间后，机器人的路线会相对固定，则机器人无法对环境进行有效的探索。

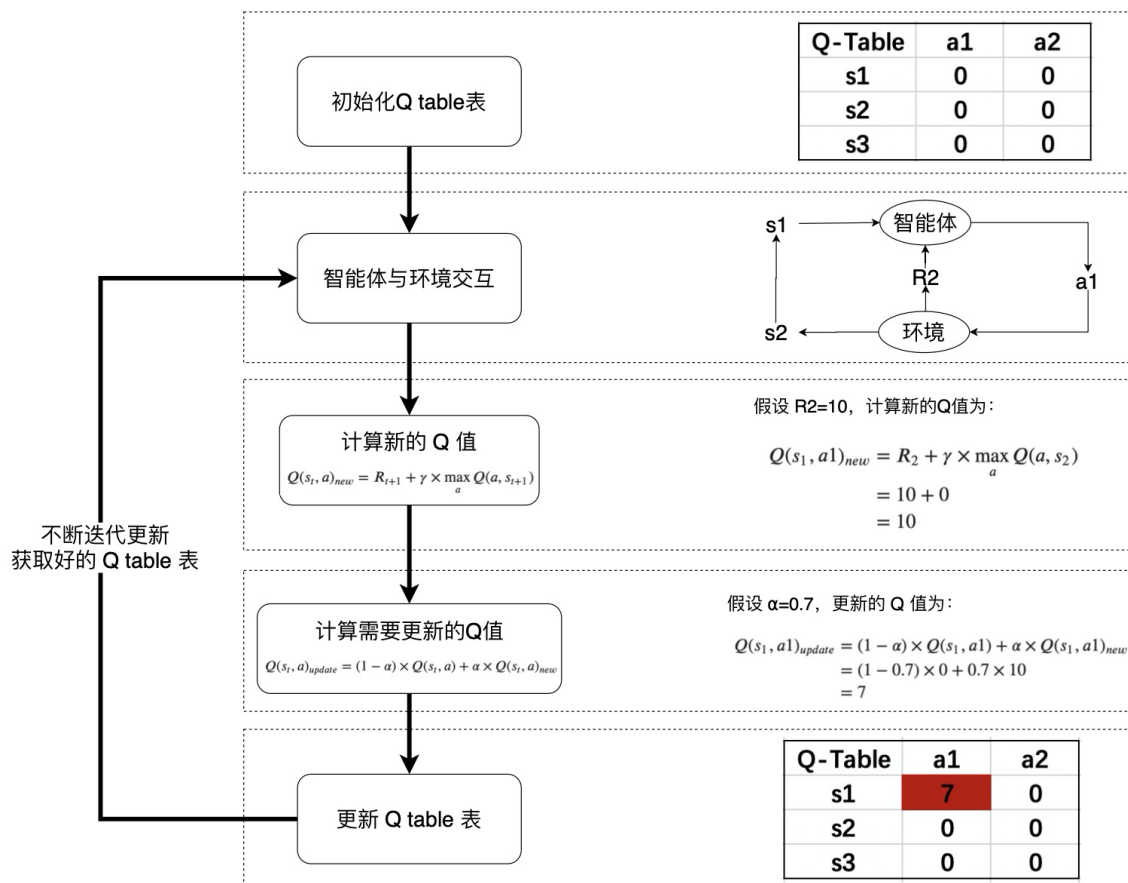
因此需要一种办法，来解决如上的问题，增加机器人的探索。

通常会使用 **epsilon-greedy** 算法：

1. 在机器人选择动作的时候，以一部分的概率随机选择动作，以一部分的概率按照最优的 Q 值选择动作。
2. 同时，这个选择随机动作的概率应当随着训练的过程逐步减小。



Q-Learning 算法的学习过程



Robot 类

在本作业中提供了 QRobot 类，其中实现了 Q 表迭代和机器人动作的选择策略，可通过 `from QRobot import QRobot` 导入使用。

QRobot 类的核心成员方法

1. `sense_state()`: 获取当前机器人所处位置
return: 机器人所处的位置坐标，如: (0, 0)
2. `current_state_valid_actions()`: 获取当前机器人可以合法移动的动作
return: 由当前合法动作组成的列表，如: ['u','r']
3. `train_update()`: 以训练状态，根据 QLearning 算法策略执行动作
return: 当前选择的动作，以及执行当前动作获得的回报, 如: 'u', -1
4. `test_update()`: 以测试状态，根据 QLearning 算法策略执行动作
return: 当前选择的动作，以及执行当前动作获得的回报, 如: 'u', -1
5. `reset()`
return: 重置机器人在迷宫中的位置

DQN 算法介绍

强化学习是一个反复迭代的过程，每一次迭代要解决两个问题：给定一个策略求值函数，和根据值函数来更新策略。而 DQN 算法使用神经网络来近似值函数。[\(DQN 论文地址\)](#)

• DQN 算法流程

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M do

Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ do

With probability ϵ select a random action a_t

otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

Execute action a_t in emulator and observe reward r_t and image x_{t+1}

Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

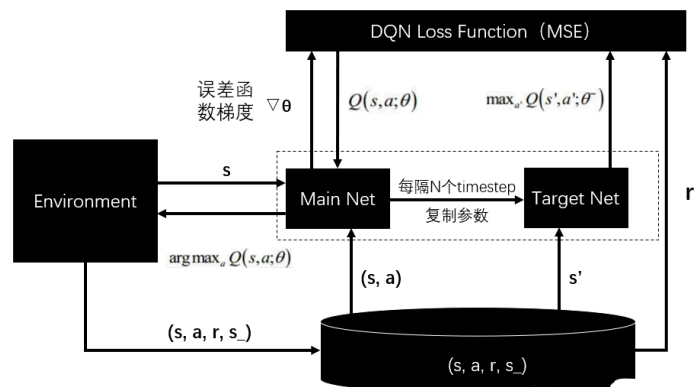
Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

Every C steps reset $\hat{Q} = Q$

End For

End For

• DQN 算法框架图



完成 DQN 算法

ReplayDataSet 类的核心成员方法

- `add(self, state, action_index, reward, next_state, is_terminal)` 添加一条训练数据
 - state: 当前机器人位置
 - action_index: 选择执行动作的索引
 - reward: 执行动作获得的回报
 - next_state: 执行动作后机器人的位置
 - is_terminal: 机器人是否到达了终止节点（到达终点或者撞墙）
- `random_sample(self, batch_size)`: 从数据集中随机抽取固定batch_size的数据
 - batch_size: 整数，不允许超过数据集中数据的个数
- **`build_full_view(self, maze)`: 开启金手指，获取全图视野**
 - maze: 以 Maze 类实例化的对象

实现目标

1. 深度优先搜索 (DFS) 算法:

- 利用栈结构存储路径节点，从起点开始逐步探索迷宫，记录访问过的位置以避免重复访问。
- 当到达目标点时，利用回溯算法生成完整路径。

2. 强化学习 (DQN) 算法:

- 初始化 Q-learning 网络，用于学习状态与动作之间的 Q 值。
- 通过训练阶段，机器人不断尝试不同路径，并根据奖励函数调整策略。
- 使用贪心策略 (epsilon-greedy) 平衡探索与利用，最终找到最优路径。

3. 奖励函数设计:

- `hit_wall`: 墙壁碰撞，奖励为正值，惩罚机器人错误动作。
- `destination`: 到达目标点，给予较大的负奖励，鼓励机器人优化路径。
- `default`: 每一步的奖励设为较小的正值，用于保持路径的连续性。

实验代码

1. 深度优先搜索算法

核心代码实现如下：

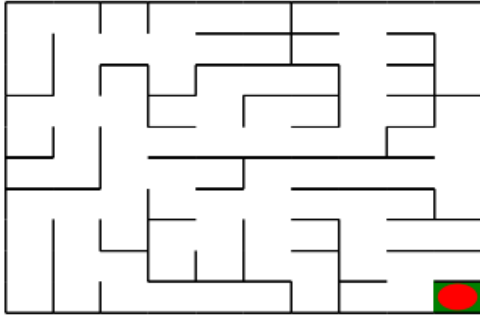
```
def my_search(maze):  
    """  
    深度优先搜索算法  
    :param maze: 迷宫对象  
    :return :到达目标点的路径 如: ["u","u","r",...]  
    """  
  
    path = []  
    stack = [] # 创建一个空的栈  
    start = maze.sense_robot()  
    root = SearchTree(loc=start)  
    stack = [root] # 根节点压入栈中  
    h, w, _ = maze.maze_data.shape  
    is_visit_m = np.zeros((h, w), dtype=np.int) # 标记迷宫的各个位置是否被访问过  
  
    while stack:  
        current_node = stack.pop() # 从栈中取出当前节点  
        is_visit_m[current_node.loc] = 1 # 标记当前节点位置已访问  
        # 到达目标点  
        if current_node.loc == maze.destination:  
            path = back_propagation(current_node)  
            break  
        # 拓展叶节点  
        if current_node.is_leaf():  
            expand(maze, is_visit_m, current_node)  
        # 将子节点入栈（逆序）  
        for child in reversed(current_node.children):  
            stack.append(child)  
    return path
```

测试代码如下

```
maze = Maze(maze_size=10) # 从文件生成迷宫  
  
path_2 = my_search(maze)  
print("搜索出的路径: ", path_2)  
  
for action in path_2:  
    maze.move_robot(action)  
  
if maze.sense_robot() == maze.destination:  
    print("恭喜你，到达了目标点")  
  
print(maze)
```

结果如下所示

搜索出的路径: ['r', 'd', 'd', 'd', 'r', 'd', 'd', 'r', 'd', 'r', 'r', 'r', 'r', 'd', 'd', 'r', 'd', 'r']
恭喜你，到达了目标点



Maze of size (10, 10)

2. 强化学习训练机器人

核心代码实现如下:

```
class Robot(TorchRobot):
    def train(self):
        """
        训练机器人，直到能够成功走出迷宫
        """
        while True:
            self._learn(batch=len(self.memory))
            success = False
            self.reset()
            for _ in range(self.maze.maze_size ** 2):
                a, r = self.test_update()
                if r == self.maze.reward["destination"]:
                    return

    def test_update(self):
        """
        测试阶段，基于 Q 值选择最佳路径
        """
        state = np.array(self.sense_state(), dtype=np.int16)
        action = self._choose_best_action(state)
        reward = self.maze.move_robot(action)
        return action, reward

    def _choose_best_action(self, state):
        """
        选择当前状态下的最优动作
        """
        state = torch.from_numpy(state).float().to(self.device)
        q_values = self.eval_model(state)
        self.eval_model.eval()
        with torch.no_grad():
            best_action_index = np.argmax(q_values)
        return self.valid_action[best_action_index]
```

实验结果

用例测试

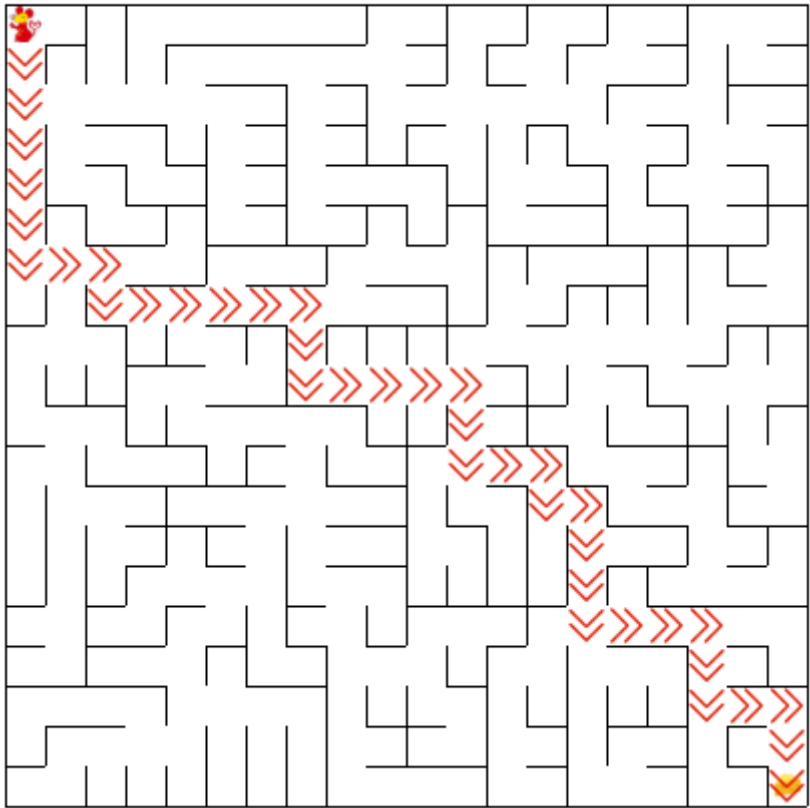
[展示迷宫](#) ▾

测试点	状态	时长	结果
测试强化学习算法 (初级)	✓	1s	恭喜, 完成了迷宫
测试基础搜索算法	✓	2s	恭喜, 完成了迷宫
测试强化学习算法 (中级)	✓	3s	恭喜, 完成了迷宫
测试强化学习算法 (高级)	✓	116s	恭喜, 完成了迷宫

1. 深度优先搜索 (DFS) 结果:

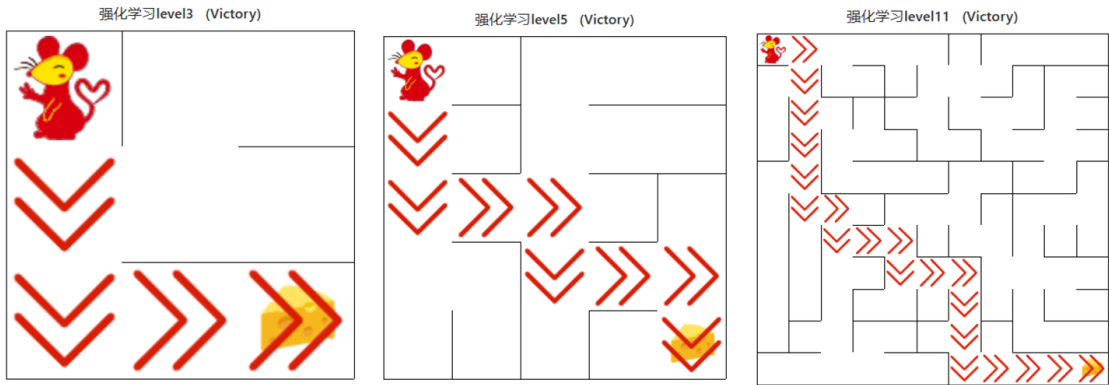
- 路径规划成功，机器人能够找到从起点到终点的路径。
- 搜索效率较低，路径不一定是最短路径。

基础搜索算法 (Victory)



2. 强化学习 (DQN) 结果:

- 机器人通过训练能够成功找到从起点到终点的最优路径。
- 随着训练的进行，机器人找到的路径逐渐缩短，效率提高。



对比分析

算法	优点	缺点
深度优先搜索	实现简单，适用于小规模迷宫	搜索效率低，路径不一定最优
强化学习 (DQN)	能够通过训练找到最优路径，适应复杂动态环境	初始训练成本高，对超参数敏感

结论

1. 深度优先搜索适用于小规模迷宫环境，算法实现简单，但对复杂迷宫不够高效。
2. 深度强化学习（DQN）算法能够通过训练找到最优路径，且适用于复杂迷宫环境，但需要较多训练时间。
3. 实验验证了强化学习在路径规划领域的潜力，同时也展示了传统算法在特定场景中的优势。

改进方向

1. 优化强化学习的超参数，如学习率、奖励函数权重。
2. 将深度优先搜索与强化学习结合，利用传统算法指导初始训练。
3. 扩展迷宫环境的复杂性，测试算法在动态迷宫中的性能。

附录

Main.py

```
# 导入相关包
import os
import random
import numpy as np
from Maze import Maze
from Runner import Runner
from QRobot import QRobot
from ReplayDataSet import ReplayDataSet
from torch_py.MinDQNRobot import MinDQNRobot as TorchRobot # PyTorch版本
```

```

from keras_py.MinDQNRobot import MinDQNRobot as KerasRobot # Keras版本
import matplotlib.pyplot as plt

import numpy as np

# 机器人移动方向
move_map = {
    'u': (-1, 0), # up
    'r': (0, +1), # right
    'd': (+1, 0), # down
    'l': (0, -1), # left
}

# 迷宫路径搜索树
class SearchTree(object):

    def __init__(self, loc=(), action='', parent=None):
        """
        初始化搜索树节点对象
        :param loc: 新节点的机器人所处位置
        :param action: 新节点的对应的移动方向
        :param parent: 新节点的父辈节点
        """

        self.loc = loc # 当前节点位置
        self.to_this_action = action # 到达当前节点的动作
        self.parent = parent # 当前节点的父节点
        self.children = [] # 当前节点的子节点

    def add_child(self, child):
        """
        添加子节点
        :param child: 待添加的子节点
        """
        self.children.append(child)

    def is_leaf(self):
        """
        判断当前节点是否是叶子节点
        """
        return len(self.children) == 0

def expand(maze, is_visit_m, node):
    """
    拓展叶子节点，即为当前的叶子节点添加执行合法动作后到达的子节点
    :param maze: 迷宫对象
    :param is_visit_m: 记录迷宫每个位置是否访问的矩阵
    :param node: 待拓展的叶子节点
    """
    can_move = maze.can_move_actions(node.loc)
    for a in can_move:
        new_loc = tuple(node.loc[i] + move_map[a][i] for i in range(2))
        if not is_visit_m[new_loc]:
            child = SearchTree(loc=new_loc, action=a, parent=node)

```

```

node.add_child(child)

def back_propagation(node):
    """
    回溯并记录节点路径
    :param node: 待回溯节点
    :return: 回溯路径
    """
    path = []
    while node.parent is not None:
        path.insert(0, node.to_this_action)
        node = node.parent
    return path

def my_search(maze):
    """
    深度优先搜索算法
    :param maze: 迷宫对象
    :return :到达目标点的路径 如: ["u","u","r",...]
    """

    path = []

    # -----请实现你的算法代码-----
    stack = [] # 创建一个空的栈
    start = maze.sense_robot()
    root = SearchTree(loc=start)
    stack = [root] # 根节点压入栈中
    h, w, _ = maze.maze_data.shape
    is_visit_m = np.zeros((h, w), dtype=np.int) # 标记迷宫的各个位置是否被访问过

    while stack:
        current_node = stack.pop() #从栈中取出当前节点
        is_visit_m[current_node.loc] = 1 # 标记当前节点位置已访问
        # 到达目标点
        if current_node.loc == maze.destination:
            path = back_propagation(current_node)
            break
        # 拓展叶节点
        if current_node.is_leaf():
            expand(maze, is_visit_m, current_node)
        # 将子节点入栈（逆序）
        for child in reversed(current_node.children):
            stack.append(child)

    # -----
    return path

import os
import random
import numpy as np
import torch
from QRobot import QRobot
from ReplayDataSet import ReplayDataSet
from torch_py.MinDQNRobot import MinDQNRobot as TorchRobot # PyTorch版本
import matplotlib.pyplot as plt

```

```

class Robot(TorchRobot):

    def __init__(self, maze):
        """
        初始化 Robot 类
        :param maze: 迷宫对象
        """
        super(Robot, self).__init__(maze)
        maze.set_reward(reward={
            "hit_wall": 10.,
            "destination": -maze.maze_size ** 2 * 10,
            "default": 1.,
        })
        self.maze = maze
        self.epsilon = 0
        """开启金手指，获取全图视野"""
        self.memory.build_full_view(maze=maze)
        self.train()

    def train(self):
        # 训练，直到能走出这个迷宫
        while True:
            self._learn(batch=len(self.memory) )
            success = False
            self.reset()
            for _ in range(self.maze.maze_size ** 2 ):
                a, r = self.test_update()
                if r == self.maze.reward["destination"]:
                    return

    def train_update(self):
        state = self.sense_state()
        action = self._choose_action(state)
        reward = self.maze.move_robot(action)

        return action, reward

    def test_update(self):
        # 获取当前状态
        state = np.array(self.sense_state(), dtype=np.int16)
        # 根据Q表选择最佳动作
        action = self._choose_best_action(state)
        # 执行动作并获得奖励
        reward = self.maze.move_robot(action)
        # 返回动作和奖励
        return action, reward

    def _choose_best_action(self, state):
        state = torch.from_numpy(state).float().to(self.device)
        q_values = self.eval_model(state) # 假设这个方法返回给定状态下所有动作的Q值
        self.eval_model.eval()
        with torch.no_grad():
            best_action_index = np.argmin(q_values)
        return self.valid_action[best_action_index] # 假设valid_actions是所有可能
动作的列表

```

